



Tecnológico de Monterrey

Reporte de modelado

Equipo 5

Nombres y Matrículas

Ulises Orlando Carrizalez Lerín	A01027715
Mónica Monserrat Martínez Vásquez	A01710965
María José Soto Castro	A01705840
Tomás Pérez Vera	A01028008
Grant Nathaniel Keegan	A01700753
Bárbara Paola Alcántara Vega	A01799609

TC3006C.101

Inteligencia artificial avanzada para la ciencia de datos II (Gpo 501)

Índice

Introducción.....	4
Mapeo de modelos.....	4
Selección de las técnicas de modelado.....	5
Modelo para la separación de vacas en el dataset (YOLOv8m).....	5
Supuestos del modelo.....	5
Diseño y ejecución de pruebas.....	5
Construcción del Modelo.....	6
Descripción del Modelo.....	7
Evaluación del modelo.....	8
Modelo para corte de imágenes (YOLOv8).....	9
Supuestos del modelo.....	10
Diseño y ejecución de pruebas.....	10
Construcción del modelo.....	10
Parámetros del modelo.....	11
Descripción del modelo.....	12
Evaluación del modelo.....	12
Modelo principal para el estimación de BCS (ConvNext).....	12
Supuestos de modelo.....	14
Cambios en la preparación de datos.....	14
Diseño y ejecución de pruebas.....	15
Conjunto de prueba y validación.....	15
Link de dataset final:.....	16
Construcción del modelo.....	16
Arquitectura del modelo final.....	16
Guardado de Modelo y Datos.....	17
Early Stopping de Modelo.....	18
ReduceLROnPlateau (TensorFlow).....	18
OneCycleR (PyTorch).....	18
Ajuste de parámetros.....	18
Construcción de modelo 1 (VGG).....	18
Arquitectura de modelo.....	18
Construcción de modelo 2 (VGG).....	20
Arquitectura de modelo.....	20
Construcción de modelo 3 (EfficientNetB7).....	21
Arquitectura de modelo.....	21
Construcción de modelo 4 (ResNet-50).....	23
Arquitectura de modelo.....	23
Construcción de modelo 5 y Final (ConvNeXt).....	25
Arquitectura de modelo.....	25
Migración a PyTorch.....	25
Descripción del modelo.....	27

Estructura y características técnicas (resumen técnico).....	28
Regularización / técnicas de aumento:.....	28
Limitaciones y riesgos.....	29
Señales de fallo del entrenamiento:.....	29
Características del modelo actual que pueden ser útiles en el futuro:.....	29
Evaluación del modelo.....	30
Criterios de éxito en minería de datos.....	30
Evaluaciones según la arquitectura.....	30
Evaluación de Robustez del Modelo.....	32
Análisis de complejidad del modelo.....	33
Curvas de aprendizaje:.....	34
Conclusión.....	36
Comparación con Profesional.....	36
Referencias.....	38

Introducción

Durante la fase de preparación de datos, se aplicaron técnicas de limpieza, filtrado y aumentación para obtener el dataset final de 628 fotografías de las vacas del CAETEC. Esta fase incluye un separador de fotografías entre carpetas con vaca y no vaca utilizando YOLOv8. Estas fotografías tienen los rasgos visuales necesarios para la detección del body condition score (BCS).

Durante la fase de modelado, el enfoque será exclusivamente sobre los datos de fotografías. Los datos tabulados (en .csv) no serán relevantes en esta fase del proyecto. Para lograr una identificación precisa del BCS, es necesario establecer qué tipo de modelo, arquitectura, y técnicas de aprendizaje serán utilizadas.

Mapeo de modelos

Modelos empleados para el procesamiento y preparación de datos			
1	YOLOv8m (vanilla)	Clasificación	Identificar si la imagen tiene vaca o no
2	YOLOv8 (re-entrenado)	Segmentación	Separación de vaca del fondo
Modelo principal			
3	ConvNext	Clasificación	Calificación por puntaje de la condición corporal (BCS)

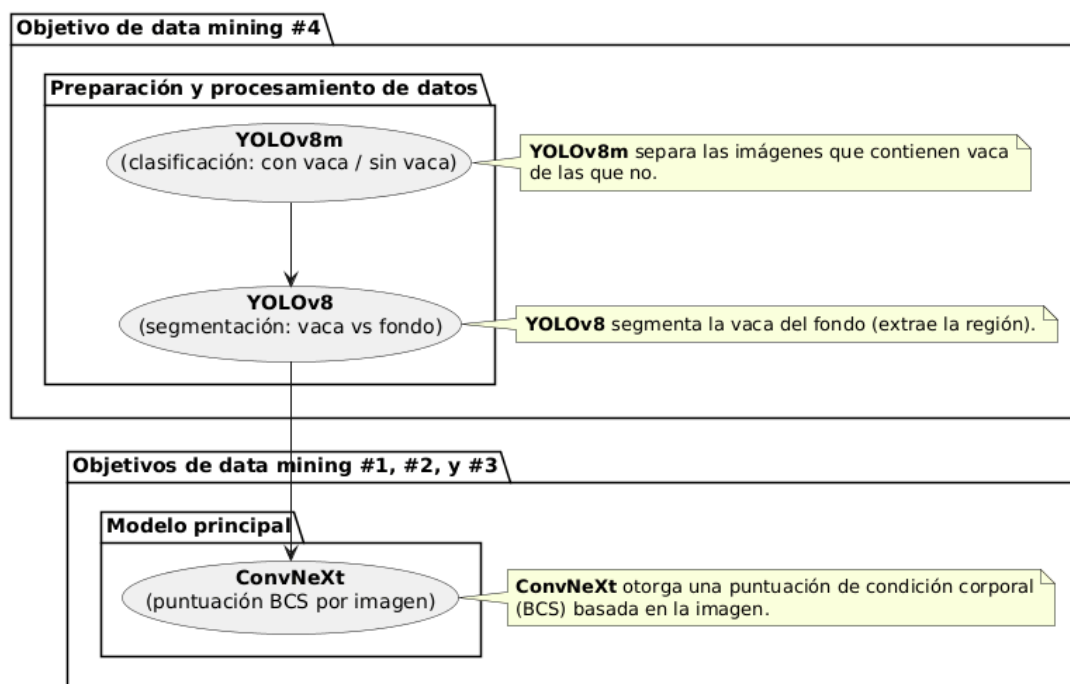


Figura 1. Gráfico del mapeo de modelos.

Selección de las técnicas de modelado

Modelo para la separación de vacas en el dataset (YOLOv8m)

El objetivo de la separación de las vacas fue aplicar una técnica de detección de objetos para agilizar la limpieza de datos. Para esto fue necesario aplicar un modelo que pudiera detectar si hay una vaca en la fotografía o no. Entre las opciones propuestas, YOLOv8 fue elegida por su precisión. Fue desarrollado por ultralytics, y entrenado sobre la base de datos de [COCO \(Common Objects in Context 2017\)](#). El modelo escogido fue yolov8m.pt. Ya que tiene un buen balance entre rendimiento y tamaño a diferencia de otros como yolov8s.pt y yolov8l.pt.

Supuestos del modelo

Los supuestos del modelo se relacionan con la base de datos en la que fue entrenado (COCO). Ya que esta nos proporciona los datos necesarios para el funcionamiento de YOLO para detección de objetos. Los supuestos están basados en [la documentación](#) y el [trabajo de investigación](#) de COCO.

Supuesto	Descripción
La clase de vacas está en COCO	COCO contiene 80 diferentes clases, es importante que la clase requerida con la que se entrenó el modelo de yolov8m se encuentre entre estas clases.
Las imágenes de la clase “cow” deberán mostrar un mínimo de rasgos identificables de vacas	Para que el modelo pueda funcionar sobre un dataset variado con vacas en diferentes posiciones, es importante que las imágenes utilizadas para el entrenamiento tengan rasgos identificables, tales como la cabeza, las piernas, los cuernos y la espalda de las vacas.

Diseño y ejecución de pruebas

Para evaluar la precisión del modelo de detección de vacas y verificar si cumple con el criterio de éxito del objetivo de minería de datos, se diseñó una prueba basada en un conjunto de referencia (“ground truth”) previamente validado de manera manual. Este conjunto consistió en 10,035 imágenes etiquetadas como “con vaca” y 37,020 imágenes etiquetadas como “sin vaca”, garantizando un 100% de exactitud en su clasificación inicial.

La prueba consistió en ejecutar el modelo con los parámetros de inferencia por defecto (IoU = 0.4, conf = 0.1) y comparar sus predicciones contra el conjunto de referencia. Para calcular el accuracy, se utilizó la relación entre el número de imágenes correctamente identificadas y el total real de imágenes etiquetadas. Específicamente, se calculó el porcentaje de aciertos mediante la fórmula:

$$Accuracy = \left(\frac{Valor\ estimado}{Valor\ real} \times 100 \right)$$

En este caso, el valor estimado corresponde al número de imágenes que el modelo clasificó correctamente como “con vaca”, mientras que el valor real corresponde al total de imágenes que realmente contenían una vaca según la clasificación manual. La comparación entre ambos valores

permite estimar la precisión del modelo y determinar si cumple con el requisito de que las imágenes seleccionadas deben contener los atributos físicos relevantes con al menos un 80% de precisión. Los resultados detallados de la ejecución se presentan en la sección de evaluación del modelo.

Construcción del Modelo

El modelo utilizado para la separación de fotografías de vacas, fue pre entrenado por ultralytics. Por lo tanto, no se utilizó una construcción del modelo por el equipo. **YOLOv8m**, una de las versiones de tamaño medio de los modelos YOLOv8, escogido por su equilibrio entre velocidad de inferencia y precisión. Para utilizar el modelo de YOLOv8m, se descargó de su repositorio de GitHub con el siguiente código y se descarga automáticamente en caso de no estar presente en la carpeta donde se ejecuta el código de limpiarVacas.py.:

- pip install ultralytics
- from ultralytics import YOLO
- model = YOLO("[yolov8m.pt](#)")

Parámetros del Modelo

A continuación, se presenta [la tabla de rendimiento](#) de los diferentes modelos de YOLOv8m, de la documentación de detección basada en COCO. Estas métricas evalúan el rendimiento de los distintos modelos de YOLOv8. El significado de los valores son:

Parámetro	Valor	Significado
Tamaño (píxeles)	640x640	Todas las imágenes de entrada tienen un valor de 640 x 640 píxeles. Si no cumplen, se redimensionan automáticamente para cumplir con el requerimiento.
mAPval 50-95	50.2	Mean Average Precision promedio entre IoU = 0.5 y 0.95. Medida de precisión global del modelo para detección de objetos.
Velocidad CPU ONIX	234.7 ms	234.7 ms. Puede procesar 4.2 imágenes por segundo usando un CPU de ONIX.
Velocidad A100 TensorRT:	1.83 ms	Puede procesar muchas más imágenes por segundo si se utiliza GPU usando A100 TensorRT.
Parámetros (Millones)	25.9M	Cantidad total de parámetros del modelo en millones. Mientras más parámetros, mejor aprendizaje, pero más tiempo de procesamiento.
FLOPs (Billions)	78.9 B	Métrica de complejidad computacional. Mientras más alto, mejor precisión en la red, pero más lenta.

Una vez que el modelo está listo para ser utilizado con el código, se puede modificar la inferencia de cómo detecta las imágenes mediante umbrales de decisión. Esto se realiza modificando los siguientes parámetros dentro del código:

Inferencia	Significado
IoU (default = 0.4)	Intersection over unión controla el manejo de diferentes detecciones solapadas sobre el mismo objeto. Más bajo implica un solapamiento más permisivo, mientras que uno más alto lo vuelve más estricto.
conf (default = 0.1)	Controla el nivel mínimo de certeza que el modelo detecta una vaca entre 0 y 1. 0.1 es un nivel de confianza bajo, lo que hace que el modelo pueda detectar imágenes donde solo se ve parte de la vaca. conf demasiado alto asume un riesgo de perder valores reales. Este valor bajo fue perfecto para el proyecto, ya que no hubo detecciones falsas de vacas en la carpeta “con vacas”.
device (default = cpu)	Especifica donde se realizarán los cálculos de inferencia. Puede ser ejecutado en cpu o GPU dependiendo de la complejidad del proyecto.

Descripción del Modelo

Para automatizar la separación de imágenes relevantes para el análisis de BCS y DEL, se utilizó el modelo de detección de objetos YOLOv8m.pt, parte de la arquitectura YOLOv8 desarrollada por Ultralytics. Este modelo fue elegido por su buen equilibrio entre precisión y eficiencia, permitiendo detectar vacas en imágenes de manera rápida y confiable.

YOLOv8m está preentrenado en el dataset COCO 2017, que incluye múltiples ejemplos de vacas, lo que le permite reconocerlas sin necesidad de un entrenamiento adicional. Durante la inferencia se usaron los parámetros por defecto (IoU = 0.4, conf = 0.1), adecuados para maximizar la identificación de imágenes que contienen animales.

Aunque YOLOv8m ofrece un buen desempeño para la detección de vacas, presenta limitaciones importantes ya que puede generar falsos negativos en condiciones visuales complejas (sombras, ángulos inusuales, obstrucciones o encuadres parciales), y su entrenamiento previo en COCO hace que no siempre se adapte a las particularidades del entorno del CAETEC. Además, su precisión depende fuertemente de los parámetros de inferencia utilizados, y aunque identifica la presencia de una vaca, no garantiza que la imagen tenga la calidad o el ángulo adecuado para evaluar correctamente el BCS. Por ello, aún se requiere revisión manual en casos límite o situaciones con condiciones poco favorables.

Evaluación del modelo

El modelo se ejecutó con los valores de inferencia default (IoU = 0.4, conf = 0.1)

Dataset	Imagenes iniciales	Imágenes totales en carpeta “sin vacas”	Número de falsos negativos dentro de “sin vacas”	Imágenes totales en carpeta “con vacas”	Accuracy final (en porcentaje)
Completo con falsos negativos agregados manualmente	47,093	37,020	0	10,035	100%
Ejecutado con falsos negativos (accuracy imperfecto)	47,093	37,842	822	9,213	91.84%

En esta prueba, podemos observar que de las 37,842 imágenes que fueron clasificadas como “sin vaca”, 822 fueron falsos negativos. calculando un porcentaje basado en el valor estimado y el valor real ($9,213 / 10,035 \times 100$), nos da un porcentaje de accuracy de 91.84%.

Los resultados obtenidos permiten concluir que el modelo cumple con el criterio de éxito establecido para el objetivo de minería de datos. El proceso automatizado logró identificar correctamente las imágenes que contienen vacas con un accuracy del 91.84%, aun considerando los 822 falsos negativos detectados en la carpeta “sin vacas”. Esto significa que, de las 10,035 imágenes reales con vacas, el modelo reconoció correctamente 9,213, lo cual representa un filtrado eficaz y suficientemente preciso para construir el dataset requerido.

Este desempeño supera el requisito de que las imágenes seleccionadas presenten los atributos físicos necesarios del animal con aproximadamente un 80% de precisión, ya que el modelo mantiene una identificación por encima del 90%, incluso bajo condiciones no ideales de inferencia.

¿Por qué el modelo no detectó 822 imágenes de las vacas?

Un aspecto importante es que las imágenes que fueron detectadas como falsos negativos, son imágenes que son difíciles de contar como una vaca por un modelo de aprendizaje de máquina. Esto es debido a condiciones de luz, la vaca ocupando demasiado espacio, o solo mostrando una diminuta parte como la cabeza o las piernas. Esto hace difícil la detección a menos que se asuma el contexto de que dichas partes o fotografías imperfectas pertenecen a una vaca. Por esto, se puede concluir que el nivel de accuracy del separador de vacas es bastante bueno cuando la vaca es claramente visible. A continuación, se mostrarán varios ejemplos de dichas imágenes que no pudieron ser detectadas.

Imagen mostrando un pequeño pedazo de las piernas de la vaca 	Imagen mostrando un pequeño pedazo de la cabeza de la vaca 	Imagen con condiciones de luz brillosas y posición cerca de la cámara 	Imagen demasiada borrosa y con suciedad 
Imagen con la vaca cubriendo la mayoría de la cámara 	Imagen con la vaca en una posición que no muestra rasgos identificables de vacas 	Imagen donde la vaca pasa demasiado rápido y la imagen sale borrosa 	

En las pruebas realizadas, alcanzó una precisión aproximada del 91.84%, cumpliendo el criterio de éxito establecido y garantizando que las imágenes seleccionadas fueran representativas para el entrenamiento del modelo principal de estimación del BCS. El modelo demostró ser robusto y eficiente en el procesamiento de datos, sin generar falsos positivos. En los criterios de los objetivos de minería de datos, se establece que para ser exitoso, los modelos del equipo deben tener un nivel de precisión de 80% como mínimo. El cual el separador de vacas superó sin problema.

Precisión Separador de vacas	91.87%	Criterio de éxito de precisión	80%
------------------------------	--------	--------------------------------	-----

El modelo basado en YOLOv8m automatiza una tarea que antes era manual y costosa: filtrar imágenes útiles para el análisis del BCS (Body Condition Score). Con este modelo, el proceso se acelera y estandariza detectar automáticamente si hay una vaca presente. Después envía las imágenes con vaca a una carpeta separada. Esto permite que el equipo ahorre tiempo, ya que la gran cantidad de imágenes tomadas no contienen vacas. El exceso de imágenes irrelevantes afecta negativamente la calidad del dataset y el desempeño de los modelos posteriores de clasificación del BCS. Por tanto, el modelo cumple con los criterios del objetivo de minería de datos planteados, al mejorar la eficiencia de filtrado y la calidad de los datos.

Modelo para corte de imágenes (YOLOv8)

Durante el proceso de análisis de los datos, se observó que las imágenes presentaban un alto nivel de ruido y que la vaca no siempre se encontraba en la misma posición al momento de la captura. Por esta

razón, fue necesario emplear un modelo capaz de identificar con precisión la ubicación de la vaca en la imagen (en términos de píxeles) y de extraer las características específicas que resultan de interés para nuestro estudio.

Para este propósito, se seleccionó el modelo YOLOv8, ya que permite detectar objetos en cualquier parte de la imagen, en este caso las vacas, y generar bounding boxes precisas que cubran el área de interés de la vaca. Esto resulta fundamental para recortar únicamente las regiones de interés utilizando OpenCV, de manera que las imágenes de entrada al siguiente modelo sean más limpias y eficientes para qué intriga la información correcta para sacar el BCS de la vaca.

Supuestos del modelo

Para garantizar un correcto funcionamiento de este modelo se plantearon varios supuestos:

Supuesto	Descripción
Mismo ángulo de fotografía.	Todas las fotografías utilizadas fueron capturadas desde un mismo ángulo y con la cámara posicionada en la parte superior. En consecuencia, el modelo fue entrenado exclusivamente con imágenes obtenidas desde esa perspectiva. Intentar ejecutar el modelo con imágenes tomadas desde un ángulo o ubicación diferente podría afectar su desempeño y generar resultados incorrectos.
Muestran los rasgos necesarios para clasificar BCS en la vaca	Antes del entrenamiento, todas las imágenes fueron revisadas por los integrantes del equipo para asegurar que cumplieran con las características necesarias para estimar adecuadamente el BCS de la vaca. De esta manera, se garantizó que el modelo pudiera aislar las características relevantes.
Presencia de la vaca en la imagen	Es importante señalar que el modelo fue entrenado únicamente con imágenes en las que la vaca estaba presente. Por lo tanto, la ausencia del animal en una imagen podría provocar un mal funcionamiento.

Diseño y ejecución de pruebas

Como parte de las pruebas para verificar el correcto funcionamiento del modelo, se ejecutó una evaluación utilizando un conjunto de 200 imágenes. En esta etapa, se realizó una verificación manual de los resultados para confirmar que el modelo generará los cortes de manera correcta y conforme a lo esperado.

Además, al finalizar el proceso de entrenamiento, se aplicó una prueba de desempeño (test) a partir de la cual se obtuvieron métricas que permitieron evaluar la precisión y efectividad del modelo.

Construcción del modelo

A diferencia del modelo anterior, este modelo requiere un proceso adicional de entrenamiento sobre la base del modelo YOLOv8 por defecto. Esto se debe a la necesidad de indicarle qué partes específicas de la vaca y de la imagen deben recortarse, con el fin de que el modelo encargado de estimar el Body

Condition Score (BCS) trabaje con imágenes más limpias y con menor nivel de ruido, facilitando así la extracción de información relevante.

Para llevar a cabo este modelo, se realizó una preparación de los datos mediante la plataforma Roboflow, en la cual se generó la clase correspondiente y se etiquetaron las características de la vaca que resultaban de interés. Esto permitió al modelo identificar dichas regiones mediante bounding boxes.

Posteriormente, se desarrolló un script en Python para reentrenar el modelo YOLOv8, utilizando únicamente la clase para aislar el área de interés. Cabe destacar que los resultados de este modelo no constituyen el objetivo final del sistema, ya que las coordenadas generadas por YOLOv8 se emplean posteriormente en OpenCV para recortar las imágenes y obtener versiones más limpias y enfocadas, que servirán como entrada para el modelo encargado de estimar el Body Condition Score (BCS).

Parámetros del modelo

Los parámetros utilizados para el reentrenamiento del modelo YOLOv8 con la clase e información personalizada fueron los siguientes:

Parámetro	Valor	Significado
Data	"{directorio de dataset}"	Directorio de dataset con etiquetas personalizadas y coordenadas de las bounding boxes en formato YOLOv8.
Epochs	200	Número de épocas de entrenamiento. Se estableció un total de 200 épocas de entrenamiento con el objetivo de permitir que el modelo convergiera adecuadamente. Este número se consideró suficiente para que el modelo aprendiera.
Batch	16	Tamaño de los lotes (batches) de imágenes empleados durante el entrenamiento. Se seleccionó un tamaño de lote de 16 imágenes como punto de equilibrio entre el uso eficiente de memoria de la GPU y la estabilidad del entrenamiento.
imgsz	640	Tamaño de las imágenes de entrada al modelo. que es el tamaño recomendado por defecto en YOLOv8 para mantener un balance entre la resolución suficiente para detectar detalles relevantes y la velocidad de procesamiento durante el entrenamiento e inferencia.
device	'0'	Especifica el dispositivo utilizado para el entrenamiento, ya sea GPU o CPU. Se especificó el valor '0' para indicar el

		uso de la GPU disponible en el sistema, con el fin de acelerar significativamente el proceso de entrenamiento
--	--	---

Este modelo se entrenó de manera local en una laptop con una GPU RTX 4050 con 8GB de VRAM.

Descripción del modelo

Este modelo se basa en la arquitectura YOLOv8 y tiene como objetivo recortar, a partir de las imágenes de entrada, la vaca y sus características más relevantes para evaluar el BCS. Esto permite mejorar la abstracción de características del BCS-Classfier y reducir el ruido, de modo que pueda obtener mejores resultados al predecir el puntaje de condición corporal (BCS) de cada vaca.

Evaluación del modelo

Para evaluar su desempeño se utilizó la métrica mAP (mean Average Precision), que mide la precisión promedio de las detecciones. En este caso, se empleó el indicador mAP@50–95 (B), el cual obtuvo un valor de 80.1 %, resultado considerado aceptable para el objetivo de este modelo, ya que indica que el corte conserva adecuadamente la región de interés sin pérdida significativa de información, considerando que cumplió el criterio de éxito de objetivo de data mining, el cual dice que “Las imágenes seleccionadas para entrenar el modelo presentan los atributos físicos de la vaca con aproximadamente un 80% de precisión.”. A continuación se muestra un ejemplo de los resultados:

Entrada	Resultado
	

Modelo principal para el estimación de BCS (ConvNext)

Para alcanzar los resultados establecidos en el objetivo de negocio y data mining, se identificó la necesidad de clasificar las imágenes obtenidas en el CAETEC, a partir del Body Condition Score (BCS). Para ello, se iteró por distintas arquitecturas (CNN/VGG, efficientnetB7, ResNet-50) hasta llegar a ConvNext-Base, debido a su capacidad para extraer características relevantes de datos con pocos elementos, como fue el caso de 628 imágenes.

La arquitectura es más reciente que ResNet-50, y aunque presentan resultados similares, ConvNext-Base surgió con el objetivo de modernizar las redes neuronales convolucionales (CNN) para acercarse a un rendimiento nivel transformador (Transformer-level performance, ViT). Tiene un mejor desempeño en datasets de menor tamaño y permite alcanzar mayor precisión si se puede permitir el cómputo. Además, permite utilizar imágenes de mayor tamaño para la extracción de características con un DownSampling atrasado y mapas de característica más gruesos, permitiendo 89 millones de parámetros (Liu et al., 2022).

Adicionalmente, fue necesario elegir una plataforma adecuada para entrenar el modelo. Al principio se utilizó **TensorFlow** debido a su alto grado de documentación en la web, sin embargo, tras presentarse problemas de compatibilidad con **Keras** se migró la arquitectura planeada al framework **PyTorch** para el desarrollo del proyecto, ya que es más simple de utilizar y presenta menos problemas de establecimiento de conexión con CUDA para entrenamiento en GPU además de estar estableciéndose como el nuevo estándar más utilizado en la industria.

Adicionalmente, la **regresión lineal** permite generar una predicción en la puntuación del BCS de una vaca en base a una imagen, siguiendo las categorías por cuartiles de las puntuaciones estándar (1 al 5 en cuartiles a modo: 1, 1.25, 1.5, 1.75, 2, 2.25, 2.5... etc.) sin necesidad de depender de una cantidad grande de clases (en este caso 20 clases por puntuación), lo cual dificultaría el entrenamiento debido a que la base de datos de imágenes no cuenta con una cantidad suficiente de vacas por cada clase (puntaje), lo cual provocaría un sesgo en la clasificación de cada una, sin mencionar que muchas imágenes pueden caer en categorías intermedias entre las clases, donde clasificar el mismo tipo de imagen en dos clases en vez de una, podría provocar un error más grande.

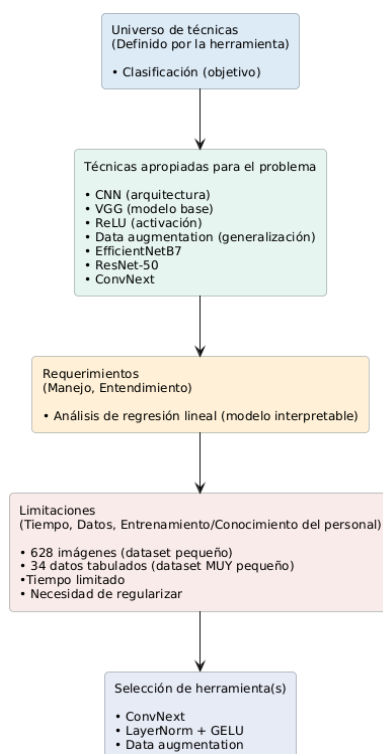


Figura 2. Proceso de selección del modelo.

Supuestos de modelo

Para garantizar la consistencia entre las fases de entendimiento y preparación de datos comparado con la fase de modelado, es importante definir una serie de supuestos sobre las fotografías utilizadas como entrada del modelo. Estos supuestos establecen los criterios mínimos de calidad y formato que deben cumplirse para garantizar un desempeño adecuado del modelo de aprendizaje profundo. Para establecerlos, se revisaron los documentos de fases previas. A continuación, se presentan los principales supuestos considerados para el conjunto de imágenes:

Supuesto	Descripción
Mismo ángulo de fotografía.	Todas las fotografías fueron tomadas desde el mismo ángulo y posición superior de la cámara. En la preparación de datos, todas las fotografías de vacas que fueron tomadas desde el ángulo lateral fueron filtradas del dataset.
Muestran los rasgos necesarios para clasificar BCS en la vaca	Todas las fotografías de vacas tienen visibles los rasgos anatómicos necesarios para poder clasificarlas en base a su BCS. Mostrando los huesos ilíacos, isquiones y ligamentos coxígeos. Fotografías donde no aparecen (por ejemplo donde solo se muestran la espalda o las piernas) fueron filtradas. De igual forma, imágenes donde no aparece ninguna vaca fueron filtradas. Y donde todas las demás condiciones son cumplidas, pero por suciedad en la cámara, o cualquier otra condición que opaque o cubra los rasgos de la vaca son filtradas.
No aparecen imágenes repetidas	En el dataset, todas las imágenes son únicas. Aunque algunas son muy similares debido a que fueron tomadas con segundos de diferencia entre cada fotografía.
Las imágenes no son borrosas	Las imágenes borrosas, donde la vaca pasa demasiado rápido, o no se ve claramente fueron filtradas. A pesar de mostrar la posición y rasgos correctos. Esto evita que el modelo se confunda, ya que debe existir claridad de los rasgos físicos para identificar el BCS de las vacas.
Todas las vacas tienen una clasificación de BCS entre 2 y 5	Es importante considerar que el rango de los datos de clasificación BCS, es entre 2 y 5 puntos. Lo cual deja las clases (1, 1.25, 1.50 y 1.75) como faltantes.
Luz Adecuada	Todas las imágenes tienen un brillo adecuado donde se pueden definir los rasgos necesarios para clasificar las vacas. Fotos demasiado oscuras o brillantes fueron filtradas en la fase previa.

Cambios en la preparación de datos

La fase de modelado comenzó con el set inicial de 628 fotografías. Durante el desarrollo del proyecto, el único cambio realizado es sobre el recorte de las imágenes. Hubo fotografías que presentaban problemas para la clasificación de las vacas. Por lo que se tomó la decisión de regresar al dataset y aplicar el código de YOLO Cropper a las imágenes que presentaban problemas. Los resultados se presentaron en el [Reporte de preparación de datos](#) en la sección de “Corrección Crop imágenes”

Diseño y ejecución de pruebas

Se necesita crear una prueba por cada criterio de éxito. En el modelo se implementó con la técnica de PyTorch `model.train()`, donde se configura el modelo para ser entrenado.

Función	Descripción	Criterio de éxito
<code>mean_absolute_error(yTrue, yPred)</code>	Mean Absolute Error, representa el promedio de error absoluto, evitando los valores negativos y positivos.	Mean Absolute Error (MAE) ≤ 0.4
<code>compute_bias_variance_from_ensemble(preds_ensemble, y_true)</code>	El bias representa el promedio de error en las predicciones	Bias ± 0.5 de BCS
	La varianza representa la dispersión de los datos	Varianza donde $0.05 < \text{predicción} < 0.3$
<code>r2_score(yTrue, yPred)</code>	R cuadrada representa el nivel de exactitud del modelo al predecir el valor BCS	$R^2 \geq 70\%$

Dichas métricas se ejecutan en el código: [computed_detailed_metrics.py](#)

Conjunto de prueba y validación

El modelo se probará con datos no vistos durante el entrenamiento, garantizando una evaluación imparcial del desempeño.

De acuerdo a las 628 imágenes con las que se contaban, se dividió el dataset en train/validation/test en fracción 80/10/10. Esto nos deja con aproximadamente 503 imágenes en entrenamiento, 62 imágenes de validación y 62 en pruebas. Para ello se utilizó el código: [split_dataset.py](#)

Primero el script lee un directorio que contiene subcarpetas, donde cada subcarpeta representa una clase. Luego valida que cada archivo tenga un formato de imagen permitido (.jpg, .jpeg, .png, etc.). Posteriormente divide las imágenes de cada clase en tres conjuntos: 80% para entrenamiento, 10% para validación y 10% para pruebas. Finalmente crea las carpetas correspondientes (train/val/test) y coloca en cada una las imágenes que pertenecen a cada clase, manteniendo la estructura organizada del dataset.

Para dividir las imágenes entre las tres particiones, se utilizó un procedimiento de muestreo dividido por clase. Esto significa que las imágenes de cada clase se repartieron manteniendo la proporción original de clases en cada conjunto. De esta forma, todas las clases del dataset se encuentran representadas tanto en entrenamiento como en validación y pruebas, evitando sesgos hacia alguna categoría en particular. La división se realizó usando la función `train_test_split` de scikit-learn, garantizando reproducibilidad mediante la fijación de una semilla aleatoria. Este código, no garantiza que todas las clases se puedan representar en train/validation/test por igual. Pero debido a que en el dataset se por clase hay mínimo 37 imágenes por clase, se tienen mínimo entre 3 y 4 imágenes de cada clase por test y validation.

En cuanto la transformación y aumentación de datos para el entrenamiento se desglosa de la siguiente manera:

Transformación	¿Qué hace?
RandomResizedCrop	Recorta aleatoriamente parte de la imagen y la redimensiona.
RandomHorizontalFlip	Invierte la imagen horizontalmente.
RandomApply(ColorJitter)	Cambia brillo, contraste, saturación y tono aleatoriamente.
RandomRotation(15)	Rota la imagen ± 15 grados.
ToTensor	Convierte a tensor.
Normalize	Normaliza los canales RGB.

Link de dataset final:

📁 Uncropped and classified (do not eliminate)

Construcción del modelo

Arquitectura del modelo final

ConvNeXt-Base es una arquitectura de red neuronal convolucional (CNN) moderna cuya arquitectura sigue una estructura jerárquica de cuatro etapas, en las cuales la resolución espacial se reduce progresivamente mientras aumenta la dimensionalidad de los canales ($96 \rightarrow 192 \rightarrow 384 \rightarrow 768$). Inicia con un stem de tipo “patchify”, con una convolución de 4×4 con paso (stride) 4 que reemplaza la convolución de 7×7 y la capa de maxpooling típicas de ResNet, generando parches no superpuestos similares a los tokens utilizados en los transformadores.

Cada etapa contiene varios bloques ConvNeXt, compuestos por una convolución separable de 7×7 (que captura dependencias espaciales de largo alcance), seguida de una normalización por capas (Layer Normalization), una convolución puntual de 1×1 que expande las características cuatro veces, una activación GELU, y una última convolución de 1×1 que proyecta de nuevo a la dimensión original. Se mantienen las conexiones residuales para estabilizar el flujo del gradiente, pero se sustituyen BatchNorm y ReLU por LayerNorm y GELU, lo que proporciona una optimización más estable y una mayor capacidad representacional. El uso de núcleos grandes, inverted bottlenecks, simplificación del muestreo espacial y normalización tipo transformador, permiten conservar la eficiencia inductiva de las convoluciones, al mismo tiempo que se alcanzan niveles de rendimiento comparables con arquitecturas basadas en self-attention (Liu et al., 2022).

Como función de pérdida se considera óptimo emplear el Mean Absolute Error (MAE), ya que el problema se plantea como una regresión lineal. Finalmente, se propone utilizar el optimizador Adam por su eficiencia en el ajuste de los parámetros del modelo. La arquitectura final se vería de la siguiente manera:

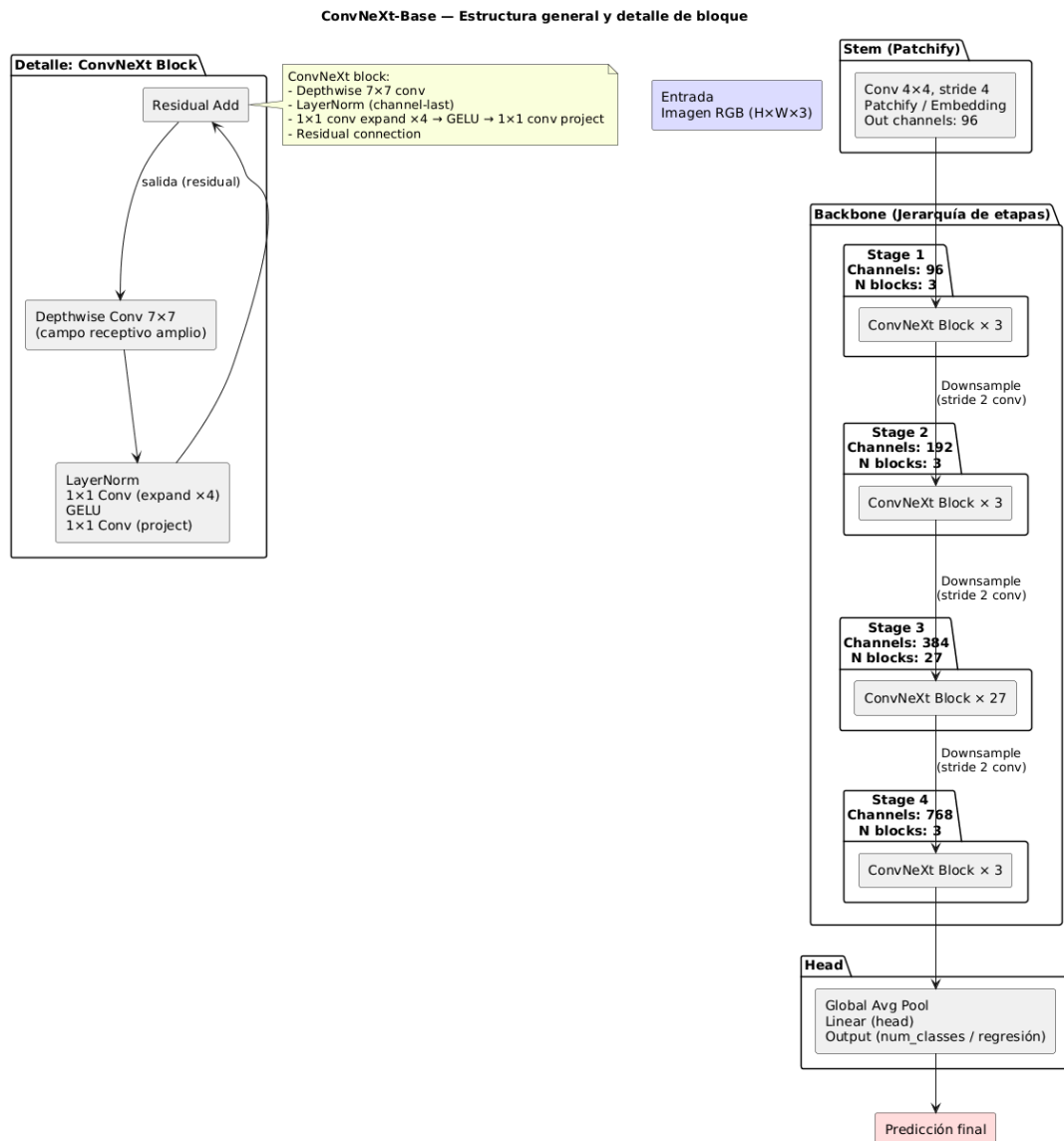


Figura 3. ConvNeXt-Base Estructura general y detalle de bloque.

Guardado de Modelo y Datos

Para asegurar que el progreso realizado en el modelo no se ha perdido en caso de una falta de memoria, error u otra circunstancia, se decidió implementar la técnica ModelCheckpoint. Durante la fase de entrenamiento, el modelo solo se guardará cuando los pesos mejoren en base a la técnica de medición de MAE (Mean Absolute Error), sólo el mínimo error se volverá a guardar.

Al finalizar el entrenamiento, el modelo se vuelve a guardar, pero esta vez se guarda toda la arquitectura, pesos y proceso de entrenamiento por medio de la función de PyTorch model.save().

Early Stopping de Modelo

Para asegurar que el modelo está corriendo y mejorando continuamente, se implementó una función de early stopping de PyTorch, donde si el modelo no presenta alguna mejora en cierto número de épocas en la técnica de monitoreo (MAE), el modelo revertirá sus pesos a los que presentan mejor desempeño y cortará su entrenamiento.

ReduceLROnPlateau (TensorFlow)

Para evitar que el modelo se pare automáticamente, debido a Early Stopping. Se decidió incorporar una reducción del margen de aprendizaje (learning rate), por medio de la función ReduceLROnPlateau de PyTorch, en el cual si el modelo no presenta alguna mejora en un cierto número de épocas, se reduce en cierto porcentaje. Así la mejora del modelo sigue ocurriendo y se aprovecha mejor el tiempo.

OneCycleR (PyTorch)

Para acelerar la convergencia del modelo y mejorar su capacidad de generalización, se decidió incorporar la estrategia OneCycleLR de PyTorch. Este método ajusta el aprendizaje (learning rate) siguiendo un único ciclo: primero incrementándose hasta un valor máximo y posteriormente reduciéndolo de manera progresiva hasta valores muy bajos. Este comportamiento controlado permite que el modelo explore más ampliamente durante las primeras iteraciones y refine con mayor precisión en las etapas finales, aprovechando mejor el tiempo de entrenamiento y favoreciendo resultados más estables.

Ajuste de parámetros

Construcción de modelo 1 (VGG)

Arquitectura de modelo

VGG16 es una red convolucional de 16 capas caracterizada por su arquitectura simple y uniforme, basada en pilas de convoluciones 3×3 y max-pooling 2×2 . Mantiene el mismo tamaño de filtro en todas las capas y aumenta progresivamente la cantidad de canales en cada bloque (64, 128, 256, 512, 512). Después de cinco bloques convolucionales, la red termina con tres capas fully connected y una capa de salida.

<i>Parámetros de Arquitectura de Modelo (iteración no. 1)</i>		
Variable	Parámetro	Justificación
epochs	30	Determinado por previa experiencia
learning_rate	0.0001	Determinado por previa experiencia
optimizer	adam	Optimizador más eficiente
dropout	0.25 y 0.4	Se añadió una capa a la

		arquitectura de dropout para evitar overfitting.
<i>Parámetros de Separación de Dataset</i>		
seed	42	número random, 42 siendo un número frecuente para hacer la separación de dataset.
batchSize	32	cantidad de imágenes para cargar dataset
valTestSplit	0.3	mantener el dataset dividido en 70% train, 15% val y 15% test
imageSize	(512, 512)	Se reduce la imagen para tener menos parámetros dentro del modelo.
shuffle	True	Revuelve las imágenes para que todos los grupos (train, val, test) tengan la mejor posibilidad de tener de todas las clases.
<i>Parámetros de Guardado de Modelo</i>		
save_weights_only	True	Cuidar almacenamiento sin tener que guardar todo el modelo
save_best_only	True	Evitar guardar pesos que no son los mejores en el modelo.
monitor	val_mae	Técnica de medición
mode	min	Vinculado con la variable de monitor, entre menor sea el error mejor.
<i>Parámetros de Early Stopping</i>		
patience	15	15 épocas representa un 10% de las épocas del modelo. Por lo que no se prolonga demasiado en parar el entrenamiento.
optimizer	adam	Optimizador seleccionado
loss	mae	Técnica de medición

Construcción de modelo 2 (VGG)

Arquitectura de modelo

Red neuronal convolucional compuesta de tres bloques de extracción de características. Cada bloque convolucional se compone de dos capas de convoluciones bidimensional con kernels de tamaño 3x3, función de activación ReLU y padding igual, seguidas por una capa de MaxPooling bidimensional de tamaño 2x2 y una capa de Dropout para regularización, con tasa de 0.25 en los dos primeros bloques y 0.4 en el tercero. Los bloques incrementan progresivamente el número de filtros, partiendo de 32 en el primero; 64 para el segundo; y 128 en el tercero. Una capa Flatten alimenta un clasificador compuesto de dos capas densas con función de activación ReLU, la primera conformada de 256 perceptrones y la segunda por 128, ambas separadas por una capa de Dropout con tasa de 0.4, concluyendo con una capa desea de 1 unidad con activación lineal.

<i>Parámetros de Arquitectura de Modelo (iteración no. 2)</i>		
Variable	Parámetro	Justificación
epochs	200	Se aumentaron debido a que con early stopping no se completarán los 200.
learning_rate	0.0001	Determinado por previa experiencia
optimizer	adam	Optimizador más eficiente
dropout	0.25 y 0.4	Se añadió una capa a la arquitectura de dropout para evitar overfitting.
<i>Parámetros de Separación de Dataset</i>		
seed	42	número random, 42 siendo un número frecuente para hacer la separación de dataset.
batchSize	32	cantidad de imágenes para cargar dataset
valTestSplit	0.3	mantener el dataset dividido en 70% train, 15% val y 15% test
imageSize	(256, 256)	Se reduce la imagen para tener menos parámetros dentro del modelo.
shuffle	True	Revuelve las imágenes para que todos los grupos (train, val, test) tengan la mejor

		posibilidad de tener de todas las clases.
<i>Parámetros de Guardado de Modelo</i>		
save_weights_only	True	Cuidar almacenamiento sin tener que guardar todo el modelo
save_best_only	True	Evitar guardar pesos que no son los mejores en el modelo.
monitor	val_mae	Técnica de medición
mode	min	Vinculado con la variable de monitor, entre menor sea el error mejor.
<i>Parámetros de Early Stopping</i>		
patience	15	15 épocas representa un 10% de las épocas del modelo. Por lo que no se prolonga demasiado en parar el entrenamiento.
optimizer	adam	Optimizador seleccionado
loss	mae	Técnica de medición

Construcción de modelo 3 (EfficientNetB7)

Arquitectura de modelo

La arquitectura entrena un conjunto (ensemble) de tres modelos idénticos, cada uno basado en la arquitectura EfficientNetB7 preentrenada en ImageNet y configurada para regresión (predicción de un valor BCS continuo). Cada modelo utiliza una cabeza de clasificación modificada que consiste en una capa GlobalAveragePooling2D, una capa Dense de 512 neuronas con activación swish y Dropout(0.3), finalizando con una capa Dense(1) con activación lineal (para la regresión). El entrenamiento se realiza en precisión mixta (FP16), aplicando estrategias avanzadas como aumento extremo de datos (RandomFlip, Rotation, Zoom, Contrast, Brightness), cálculo de pesos balanceados por clase, y un scheduler de tasa de aprendizaje Warmup Cosine Decay. Los modelos se optimizan con Adam y la función de pérdida Huber Loss (con $\delta=0.5$), y se evalúan por MAE (Error Absoluto Medio). Finalmente, el conjunto se evalúa promediando las predicciones de los tres modelos sobre el conjunto de prueba, y se genera automáticamente un reporte PDF con las métricas y gráficos de desempeño.

<i>Parámetros de Arquitectura de Modelo (iteración no. 3)</i>		
Variable	Parámetro	Justificación

epochs	150	Se redujeron las épocas de entrenamiento para evaluar la abstracción del modelo sin early stopping
learning_rate	0.0001	Determinado por previa experiencia
optimizer	adam	Optimizador seleccionado
Dropout	0.25 y 0.4	Se añadió una capa a la arquitectura de dropout para evitar overfitting.
<i>Parámetros de Separación de Dataset</i>		
seed	42	número random, 42 siendo un número frecuente para hacer la separación de dataset.
batchSize	32	cantidad de imágenes para cargar dataset
valTestSplit	0.3	mantener el dataset dividido en 70% train, 15% val y 15% test
imageSize	(612, 612)	Se incrementa para aumentar de nuevo los parámetros, y mejor resolución
shuffle	True	Revuelve las imágenes para que todos los grupos (train, val, test) tengan la mejor posibilidad de tener de todas las clases.
<i>Parámetros de Guardado de Modelo</i>		
save_weights_only	True	Cuidar almacenamiento sin tener que guardar todo el modelo
save_best_only	True	Evitar guardar pesos que no son los mejores en el modelo.
monitor	val_mae	Técnica de medición
mode	min	Vinculado con la variable de monitor, entre menor sea el error mejor.
<i>Parámetros de Early Stopping</i>		

patience	15	Se optó por quitar el early stopping, con el fin de evaluar si el modelo es capaz de obtener una mayor capacidad de abstracción.
optimizer	adam	Optimizador seleccionado
loss	mae	Técnica de medición

Construcción de modelo 4 (ResNet-50)

Arquitectura de modelo

ResNet-50 es una red convolucional de 50 capas que utiliza bloques residuales con *skip connections* para evitar el problema del *vanishing gradient* y permitir entrenar redes muy profundas. Cada bloque es de tipo *bottleneck*, compuesto por convoluciones 1×1 , 3×3 y 1×1 que reducen, procesan y expanden canales de manera eficiente. La arquitectura se organiza en cuatro etapas con 3, 4, 6 y 3 bloques, precedidas por una convolución inicial y seguidas de *global average pooling* y una capa fully connected. Es una arquitectura estable, eficiente y ampliamente usada en tareas de visión por computadora.

Parámetros de Arquitectura de Modelo (iteración no. 4)		
Variable	Parámetro	Justificación
epochs	200	Se aumentaron debido a que con early stopping no se completarán los 200.
learning_rate	0.0001	Determinado por previa experiencia
optimizer	adam	Optimizador más eficiente
Parámetros de Separación de Dataset		
seed	42	número random, 42 siendo un número frecuente para hacer la separación de dataset.
batchSize	12	Ajuste para GPU
valTestSplit	0.3	mantener el dataset dividido en 70% train, 15% val y 15% test
imageSize	(448, 448)	Se reduce la imagen para tener menos parámetros dentro del

		modelo.
shuffle	True	Revuelve las imágenes para que todos los grupos (train, val, test) tengan la mejor posibilidad de tener de todas las clases.
<i>Parámetros de Guardado de Modelo</i>		
save_weights_only	True	Cuidar almacenamiento sin tener que guardar todo el modelo
save_best_only	True	Evitar guardar pesos que no son los mejores en el modelo.
monitor	val_mae	Técnica de medición
mode	min	Vinculado con la variable de monitor, entre menor sea el error mejor.
<i>Parámetros de Early Stopping</i>		
patience	15	Se optó por quitar el early stopping, con el fin de evaluar si el modelo es capaz de obtener una mayor capacidad de abstracción.
optimizer	adam	Reduce el sobreajuste y mejora estabilidad
loss	mae	Técnica de medición
<i>Parámetros de ReduceLROnPlateau</i>		
monitor	val_mae	Técnica de medición
patience	6	No. de épocas que se espera si no hay mejora
factor	0.5	El learning rate se reduce a la mitad
min_lr	0.0000001	si el parámetro de aprendizaje queda en min_lr se deja de aplicar la reducción.

Construcción de modelo 5 y Final (ConvNeXt)

Arquitectura de modelo

Este modelo implementa un proceso de fine-tuning para una tarea de regresión utilizando un modelo base de ConvNeXt (configurable, ConvNext_base por defecto, o ResNet50 como fallback) preentrenado. El modelo está optimizado para la velocidad y eficiencia, operando con precisión mixta (AMP) en PyTorch. La metodología clave incluye congelamiento progresivo del backbone (freeze_epochs por defecto a 3) para estabilizar el entrenamiento inicial de la capa de regresión, un scheduler OneCycleLR (configurable) para una gestión óptima de la tasa de aprendizaje, y técnicas de regularización avanzadas como MixUp y CutMix adaptadas para etiquetas continuas. La capa de regresión de salida utiliza una función de pérdida SmoothL1Loss y se optimiza con AdamW. Además, emplea un muestreo ponderado aleatorio (WeightedRandomSampler) para manejar el desequilibrio de clases, asegurando una robusta predicción del valor BCS.

Migración a PyTorch

Durante el proceso de evaluación del modelo, se encontraron conflictos de versiones en los cuales no éramos capaces de implementar el modelo. Por ello decidimos migrar nuestro modelo (iteración 3) a PyTorch.

Por lo cual el nombre de algunos parámetros fueron afectados por el cambio.

<i>Parámetros de Arquitectura de Modelo (iteración no. 5)</i>		
Variable	Parámetro	Justificación
<i>Parámetros de Arquitectura Modelo</i>		
epochs	40	Se mantienen 80 épocas para evaluar la capacidad de abstracción sin early stopping.
lr (learning_rate)	0.00005	Valor determinado por experiencia previa y estabilidad con AdamW.
optimizer	AdamW	Reduce el sobreajuste y mejora estabilidad en modelos grandes (ConvNeXt).
weight_decay	0.00005	Regularización estándar para AdamW en modelos de gran escala.
betas	(0.9, 0.999)	Control de momentos del optimizador; valores estándar en AdamW.
eps	1e-7	Estabilidad numérica en

		actualizaciones de pesos.
loss	nn.SmoothL1Loss()	Pérdida tipo Huber, robusta frente a outliers en regresión.
dropout	0.3	Añadido en la capa final del modelo para reducir overfitting.
Parámetros de Separación de Dataset		
Variable	Parámetro	Justificación
seed	42	Reproducibilidad en la división del dataset.
batch_size	24	Tamaño óptimo para GPU RTX 4090 sin desbordar memoria.
valTestSplit	0.3	Dataset dividido 70% train, 15% val, 15% test.
img_size	(384, 384)	Reduce parámetros y mantiene resolución suficiente para ConvNeXt.
shuffle	True	Asegura mezcla aleatoria de las muestras en cada época.
Parámetros de Guardar el Modelo		
Variable	Parámetro	Justificación
save_weights_only	True	Solo se guardan pesos (.pth) para reducir tamaño.
save_best_only	True	Solo se guarda el checkpoint con menor pérdida de validación.
monitor	val_mae	Métrica principal de validación.
mode	min	Se busca minimizar el error absoluto medio (MAE).
Parámetros de Guardar el Modelo		
Variable	Parámetro	Justificación
early_stop_patience	6	Se puede quitar o dejarlo bajo para observar capacidad de abstracción sin interrupción

		prematura.
criterion	SmoothL1Loss (MAE-like)	Pérdida de tipo regresión robusta.
<i>Parámetros de Scheduler</i>		
Variable	Parámetro	Justificación
scheduler	OneCycleLR	Mejora la convergencia al ajustar dinámicamente la tasa de aprendizaje durante las épocas.
monitor	val_mae	Métrica base para decidir ajustes de LR (referencia).
factor	0.5	(Si se usa ReduceLROnPlateau) reduce LR a la mitad cuando no hay mejora.
min_lr	1e-7	Límite inferior para el LR.
<i>Parámetros Adicionales de Hardware</i>		
Variable	Parámetro	Justificación
use_amp	True	Usa Automatic Mixed Precisión para acelerar el entrenamiento en RTX 4090.
num_workers	8	Acelera la carga de datos en GPUs con alto rendimiento.
freeze_epochs	3	Congela capas base de ConvNeXt para estabilizar el entrenamiento inicial.
pin_memory	True	Optimiza transferencia de datos CPU→GPU.

Descripción del modelo

El código del modelo implementa un pipeline de fine-tuning para tareas de regresión sobre imágenes usando ConvNeXt-Base (con timm) como backbone por defecto y ResNet-50 como fallback. El objetivo es ajustar un modelo convolucional pre entrenado para predecir un único valor numérico por imagen (salida escalar). Está optimizado para una GPU RTX 4090, y contiene utilidades habituales de prácticas de entrenamiento avanzado, como mixed precisión (AMP), aumentos fuertes, MixUp/CutMix adaptados a etiquetas continuas, congelado inicial del backbone, scheduler OneCycleLR o Cosine, y guardado de checkpoints y predicciones.

Como el dominio es visual y similar a ImageNet (texturas, objetos, variabilidad moderada), ConvNeXt-Base como backbone pre entrenado permite obtener mejores representaciones que ResNet-50, por lo que es razonable esperar mejoras notables en métricas, especialmente cuando se cuenta con suficientes muestras y buenos aumentos. En clasificación, **ConvNeXt-Base alcanza ~85–86% top-1** en ImageNet (para dar una idea de su capacidad). En regresión, traducir esto a MAE no es directo pero ConvNeXt-Base suele ofrecer mejor capacidad representacional que ResNet-50 en transfer learning si se cuenta con suficientes datos.

Estructura y características técnicas (resumen técnico)

- Backbone: convnext_base vía `timm.create_model(name, pretrained=True, num_classes=1)`. Si `timm` no está disponible, usa `torchvision.models.resnet50` con una cabeza regresora (MLP: `Linear` → `ReLU` → `Dropout` → `Linear`).
- Cabeza (head): salida de 1 nodo (regresión). Para ResNet: `fc` reemplazada por `Linear(in_f,512)` → `ReLU` → `Dropout(0.3)` → `Linear(512,1)`. Para ConvNeXt, `Timm` construye la cabeza adaptada.
- Normalización / activación: depende del backbone (ConvNeXt usa `LayerNorm` + `GELU` internamente; ResNet usa `BatchNorm` + `ReLU`).
- Pérdida: `nn.SmoothL1Loss()` (robusta a outliers relativos a L1 y L2).
- Optimizador: `AdamW` con `weight_decay` configurable.
- Scheduler: `OneCycleLR` (por batch) o `CosineAnnealingLR`.
- Precision: `Mixed Precision` (`autocast` + `GradScaler`) opcional (`--use_amp`).
- Métricas: guarda MAE, RMSE y R^2 (función `compute_metrics` usa `sklearn`).
- Salida: un escalar por imagen (tensor shape `[B,1]` → guardado como lista `preds` (predicción)).

Regularización / técnicas de aumento:

- `RandomResizedCrop`, `RandomHorizontalFlip`, `ColorJitter`, `RandomRotation`.
- Opcionales: `MixUp` (mezcla lineal de imágenes y etiquetas) y `CutMix` (pegar región de otra imagen y mezcla proporcional de etiquetas) adaptados para targets continuos.
- Opción de `WeightedRandomSampler` para balancear clases/etiquetas si hay desbalance.

Entrenamiento

- Freeze del backbone durante `freeze_epochs` iniciales (solo head entrenable), luego unfreeze total.
- Grad accumulation configurable (`grad_accum_steps`).
- Early stopping basado en `early_stop_patience` sobre MAE de validación.
- Función de pérdida: `SmoothL1` hace que el entrenamiento sea menos sensible a outliers extremos que L2, pero más estable que L1 puro.

IO y artefactos:

- Guarda `best_checkpoint_by_val_mae.pth`, predicciones `preds_val.npz` y `preds_test.npz`, `history.json`, y `final_model.pth`.

Data pipeline:

- FolderRegressionDataset espera carpetas por label; discover_label_map asume nombres de carpeta convertibles a float (por ejemplo, carpetas 2.0, 2.25, ...).

Limitaciones y riesgos

- Computacionales:
 - ConvNext es grande (muchos parámetros y FLOPs), por lo que requiere de una GPU potente y memoria, ya que con batch grande/alto el tamaño de la imagen puede consumir mucha VRAM.
- Datos:
 - Si el dataset es demasiado pequeño, el modelo puede sobreajustar pese a aumentaciones, por lo que en ese caso ResNet-50 o modelos más pequeños pueden ser preferibles.
 - El discover_label_map asume que el nombre de carpeta se convierte a float. Si las etiquetas no están en ese formato, se romperá el flujo.
- Implementación:
 - Se debe tener cuidado con total_steps si se utilizan persistent_workers o samplers personalizados.
- Sesgo/Out-of-Distribution (OOD):
 - Como todos los modelos entrenados con datos limitados, su comportamiento fuera del dominio de entrenamiento es impredecible — se requiere monitorización y detección OOD.

Señales de fallo del entrenamiento:

- Training MAE muy bajo y val MAE alto → overfitting.
- Ambas MAE altas → underfitting o problema de datos / label noise.
- Oscilaciones grandes en loss → scheduler / LR mal ajustados o problemas numéricos (verificar scaler).

Características del modelo actual que pueden ser útiles en el futuro:

- Modularidad del backbone: se puede cambiar el modelo a variantes de timm (efficientnet, swin, etc.) para experimentar sin reescribir el pipeline.
- AMP y grad accumulation: permite escalar a tamaños de imagen más grandes sin necesitar VRAM inmensa.
- MixUp/CutMix adaptados a regresión: son útiles para otros dominios donde la variable objetivo es continua y hay mezcla natural entre categorías.
- Weighted sampler: facilita manejar distribuciones de etiquetas no uniformes.
- Checkpointing y predicciones guardadas: facilita auditoría, ensambles y replicabilidad, manteniendo la evaluación de overfitting en otros scripts para facilitar las evaluaciones post-entrenamiento.
- Head MLP configurable: permite ajustar la capacidad del head sin tocar el backbone.

Evaluación del modelo

Criterios de éxito en minería de datos

<i>Criterio de éxito</i>
1) La diferencia entre MAE de entrenamiento y prueba debe ser ≤ 0.4 . 2) El bias debe estar entre ± 0.5 BCS tomando en cuenta DEL. 3) La varianza de predicciones debe estar entre 0.05 y 0.3. 4) La precisión debe ser $\geq 70\%$ de las predicciones de BCS deben coincidir con la valoración real.

Evaluaciones según la arquitectura

<i>Modelo 1 y 2 (VGG)</i>		
Validación		
<i>Criterio de éxito</i>	<i>Resultado del modelo</i>	<i>Cumplio con el criterio</i>
Mean Absolute Error (MAE) ≤ 0.4	MAE = 0.5820	No se cumplió el criterio
Bias ± 0.5 de BCS	Bias = -0.0359	Se cumplió el criterio de éxito
Varianza donde $0.05 < \text{predicción} < 0.3$	Varianza = 0.1922	Se cumplió el criterio de éxito
Prueba		
<i>Criterio de éxito</i>	<i>Resultado del modelo</i>	<i>Cumplio con el criterio</i>
Mean Absolute Error (MAE) ≤ 0.4	MAE = 1.1314	No se cumplió el criterio
Bias ± 0.5 de BCS	Bias = -1.1047	No se cumplió el criterio

Los resultados de este modelo, no fueron exitosos a nuestras métricas de minería de datos. Se presentó un índice alto de sobreajuste, como se puede ver en la diferencia de lo que causó que las métricas de entrenamiento fueran mejores a las de validación, resultando en una diferencia de 0.55 en MAE y 1.07 en el bias . A pesar de esto, se logró una varianza del modelo entre 0.05 y 0.3.

<i>Modelo 3 (efficient B7)</i>		
Validación		
<i>Criterio de éxito</i>	<i>Resultado del modelo</i>	<i>Cumplio con el criterio</i>

Mean Absolute Error (MAE) $\leq$ 0.4	MAE = 0.2027	Se cumplió el criterio
Bias ± 0.5 de BCS	Bias = -0.042	Se cumplió el criterio
Varianza donde $0.05 <$ predicción < 0.3	Varianza = 0.232	Se cumplió el criterio
Prueba		
<i>Criterio de éxito</i>	<i>Resultado del modelo</i>	<i>Cumplio con el criterio</i>
Mean Absolute Error (MAE) $\leq$ 0.4	MAE = 0.355	Se cumplió el criterio
Bias ± 0.5 de BCS	Bias = 0.2782	Se cumplió el criterio

En el modelo con arquitectura efficientB7, se cumplieron los criterios definidos, pero fue en este momento que se estaba utilizando la métrica incorrecta de Accuracy en los modelos anteriores, se actualizó a R^2 para obtener la métrica adecuada en la regresión. Volviendo a entrenar el modelo el valor de R^2 no logró subir más de 40%.

Modelo 4 (ResNet-50)		
Validación		
<i>Criterio de éxito</i>	<i>Resultado del modelo</i>	<i>Cumplio con el criterio</i>
Mean Absolute Error (MAE) $\leq$ 0.4	MAE = 0.3317	Se cumplió el criterio
Mean Squared Error (MSE)	MSE = 0.2771	Se cumplió el criterio
Bias ± 0.5 de BCS	Bias = 0.01136	Se cumplió el criterio
Varianza donde $0.05 <$ predicción < 0.3	Varianza = 0.8561	No se cumplió el criterio
R^2 (Coeficiente de determinación) $> 70\%$	$R^2 = 0.7156$	Se cumplió el criterio
Prueba		
<i>Criterio de éxito</i>	<i>Resultado del modelo</i>	<i>Cumplio con el criterio</i>
Mean Absolute Error (MAE) $\leq$ 0.4	MAE = 0.2894	Se cumplió el criterio
Mean Squared Error (MSE)	MSE = 0.3524	Se cumplió el criterio
Bias ± 0.5 de BCS	Bias = -0.0087	Se cumplió el criterio
R^2 (Coeficiente de	$R^2 = 0.6439$	No se cumplió el criterio

determinación) > 70%		
----------------------	--	--

Modelo 5 (ConvNeXt-Base)		
Validación		
<i>Criterio de éxito</i>	<i>Resultado del modelo</i>	<i>Cumplio con el criterio</i>
Mean Absolute Error (MAE) ≤ 0.4	MAE = 0.2893	Se cumplió el criterio
Mean Squared Error (MSE)	MSE = 0.2264	Se cumplió el criterio
Bias ± 0.5 de BCS	Bias = 0.0254	Se cumplió el criterio
Varianza donde $0.05 < \text{predicción} < 0.3$	Varianza = 0.232	Se cumplió el criterio
R^2 (Coeficiente de determinación) > 70%	$R^2 = 0.7676$	Se cumplió el criterio
Prueba		
<i>Criterio de éxito</i>	<i>Resultado del modelo</i>	<i>Cumplio con el criterio</i>
Mean Absolute Error (MAE) ≤ 0.4	MAE = 0.3352	Se cumplió el criterio
Mean Squared Error (MSE)	MSE = 0.2745	Se cumplió el criterio
Bias ± 0.5 de BCS	Bias = -0.0548	Se cumplió el criterio
R^2 (Coeficiente de determinación) > 70%	$R^2 = 0.7226$	Se cumplió el criterio

El modelo ConvNext permitió alcanzar las métricas objetivo, alcanzando una R^2 del 72.26%.

Evaluación de Robustez del Modelo

Tabla de Robustes del Modelo

Instances	Poca cantidad en carpeta	Imagen oscura	Imagen muy brillante	Vaca muy blanca	Vaca muy negra	Imagen con ruido	Mal recorte	Vaca haciendo pipí
Proportion of current error	0.64%	0.16%	0.32%	1.91%	1.59%	3.34%	63%	0.16%
Real error	$0.2 * 0.0064 = 0.13\%$	$0.2 * 0.0016 = 0.03\%$	$0.2 * 0.0032 = 0.06\%$	$0.2 * 0.0191 = 0.38\%$	$0.2 * 0.0159 = 0.32\%$	$0.2 * 0.0334 = 0.67\%$	$0.2 * 0.63 = 12.6\%$	$0.2 * 0.0016 = 0.03\%$

Debido a que la mayoría de las imágenes con una clasificación errónea cuentan con un mal recorte, se refinó el modelo que realiza los recortes en las imágenes, entrenando con etiquetas nuevas en imágenes, las cuales abarcan mayor área de interés para abstraer más rasgos físicos de cada vaca.

El objetivo fue incrementar el valor de la R^2 a $\geq 70\%$ de las predicciones de BCS coincidiendo con la valoración real, mientras que el resto de métricas objetivo se alcanzaron con el Modelo ConvNeXt-Base utilizando únicamente 628 imágenes.

Análisis de complejidad del modelo

Para identificar el punto donde el aumento de parámetros deja de mejorar las métricas, y con el objetivo de garantizar un equilibrio entre precisión y generalización, se realizó un análisis para validar que el modelo elegido cumple los criterios de éxito con la mínima complejidad posible, evitando el sobreajuste.

Se empleó una división del conjunto de datos en 80% para entrenamiento, 10% para validación y 10% para prueba, asegurando que la evaluación final refleje la capacidad real de generalización de los modelos. Todos los modelos se entrenaron bajo condiciones controladas, utilizando optimización AdamW, funciones de pérdida Smooth L1 Loss (para regresión), y técnicas de regularización como data augmentation, freeze/unfreeze de capas y mixed precisión (AMP) para maximizar la eficiencia en GPU (RTX 4090).

La siguiente tabla muestra un resumen comparativo de diferentes corridas de cada arquitectura de modelo empleada en torno a la elección de una arquitectura definitiva.

Modelo	N° aproximado de parámetros que permite el modelo	Métricas			Observaciones
		Validación	Prueba	Referencia	
VGG	~138 millones (VGG16)	MAE = 0.5820 Bias= -0.0359 Var = 0.1922	MAE= 1.1314 Bias= -1.1047	MAE $\leq$ 0.4 Bias $\pm$ 0.5	Alto sobreajuste con dataset limitado. Generaliza pobremente sin regularización.
EfficientNetB7	~25.6 millones	MAE = 0.2027 Bias= -0.042 Var = 0.232	MAE= 0.355 Bias= 0.2782	MAE $\leq$ 0.4 Bias $\pm$ 0.5	Escalamiento balanceado entre profundidad, resolución y anchura, buena eficiencia.

Tras incorporar la métrica de R^2 :					
Modelo	Nº aproximado de parámetros que permite el modelo	Métricas			Observaciones
		Validación	Prueba	Referencia	
ResNet-50	~66 millones	MAE = 0.3317 Bias= 0.01136 Var = 0.8561 R^2 = 0.7156	R^2 = 0.6439 Bias= -0.0087	$R^2 \geq 0.7$ Bias $\pm$ 0.5	Mejor estabilidad gracias a conexiones residuales y menor pérdida de gradiente.
ConvNeXt	~89 millones	MAE = 0.2893 Bias= 0.0254 Var = 0.232 R^2 = 0.7676	R^2 = 0.7226 Bias= -0.0548	$R^2 \geq 0.7$ Bias $\pm$ 0.5	Mantiene las convoluciones pero optimiza la normalización y el uso del espacio. Tuvo una excelente generalización.

- Los modelos más profundos (EfficientNet, ConvNeXt) mostraron curvas de aprendizaje estables, con reducción progresiva de val_loss y convergencia temprana.
- CNN/VGG alcanzó rápidamente un punto de saturación, mostrando underfitting (baja capacidad de aprendizaje global).
- ResNet50 sirvió como buen punto intermedio, pero ConvNeXt mostró el mejor equilibrio entre precisión, estabilidad y escalabilidad.
- El modelo ConvNeXt mantiene su utilidad futura como base para tareas de transfer learning, dada su estructura modular y compatibilidad con nuevas cabezas de predicción (regresión o clasificación).

Donde ConvNeXt supera a las arquitecturas tradicionales, alcanzando una mayor precisión con menor error medio absoluto (MAE) en validación.

Curvas de aprendizaje:

Se analizarán las curvas de pérdida y métricas de entrenamiento/validación para detectar sobreajuste o subajuste.

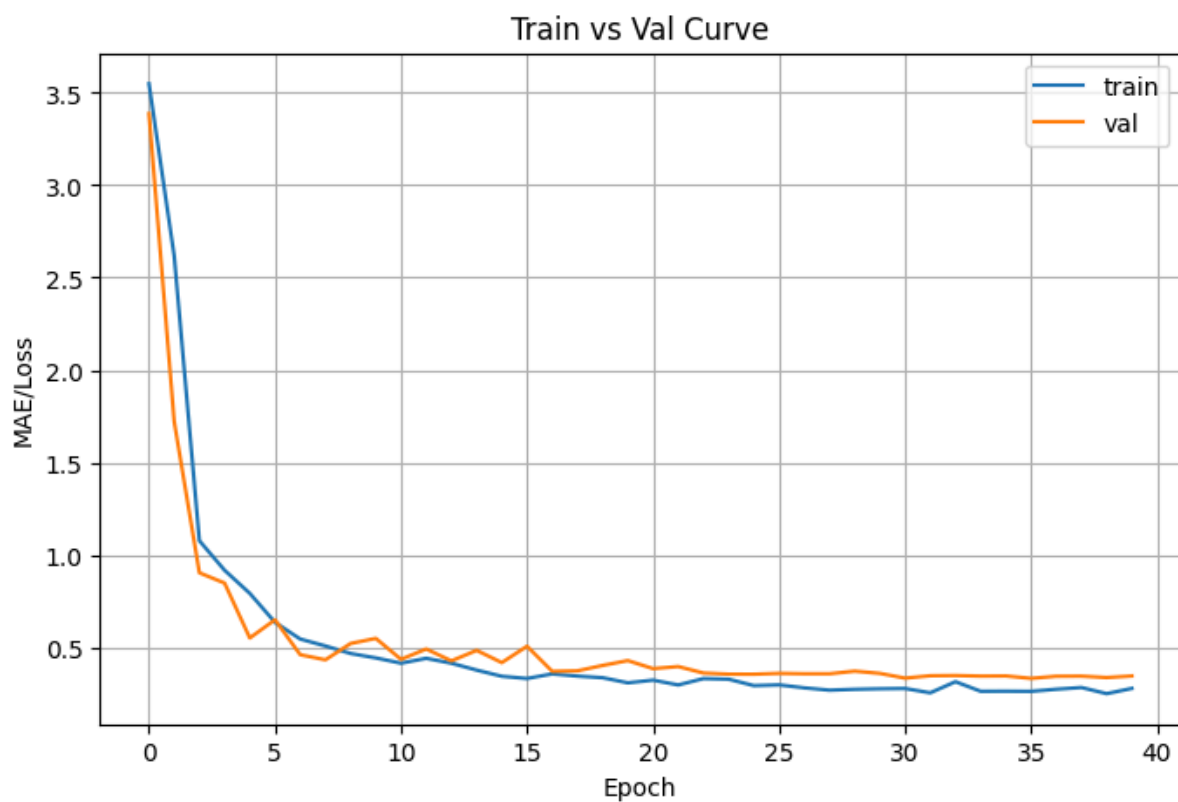


Figura 4. Curva de aprendizaje con arquitectura ResNet50

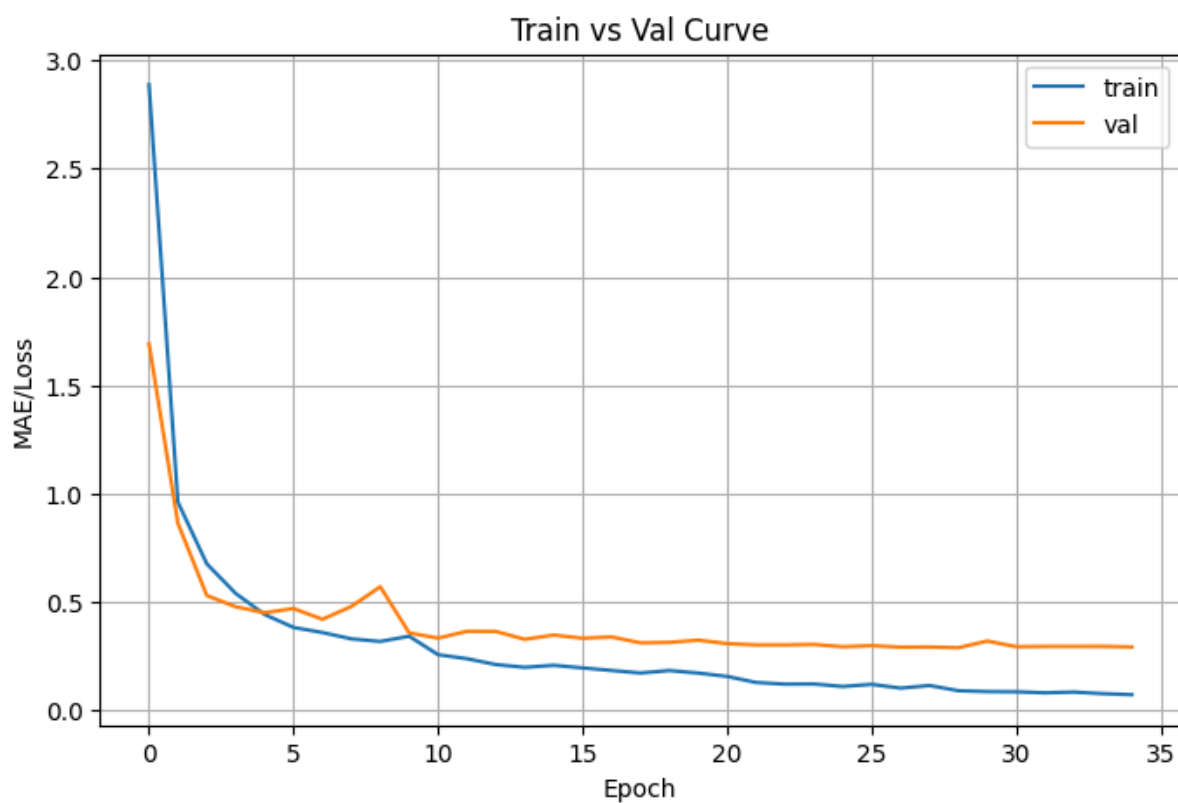


Figura 5. Curva de aprendizaje con arquitectura ConvNeXt-Base

A partir de las curvas de entrenamiento–validación y las métricas obtenidas, se observa que ResNet-50 muestra una convergencia más estable y un comportamiento más consistente entre entrenamiento y validación, lo que coincide con su menor bias y varianza moderada, además, cuenta con la ventaja inherente de las conexiones residuales para evitar pérdida de gradiente. Sin embargo, aunque su estabilidad es evidente en la gráfica—donde las curvas se mantienen próximas y sin oscilaciones marcadas—su capacidad de generalización es inferior a la de ConvNeXt-Base. En contraste, ConvNeXt presenta curvas con una ligera separación entre entrenamiento y validación, pero logra mejores métricas finales, especialmente un MAE menor y un R^2 superior tanto en validación como en prueba, indicando que el modelo abstrae de mejor manera los rasgos físicos de cada vaca acorde a su BCS. Esto sugiere que, pese a una convergencia menos suave, ConvNeXt obtiene un ajuste más predictivo y robusto, beneficiándose de optimizaciones modernas en normalización y arquitectura, lo que se refleja en su mejor capacidad de generalización frente a ResNet-50.

Conclusión

Comparación con Profesional

Los resultados obtenidos a lo largo de las distintas arquitecturas permiten concluir que el modelo ConvNeXt-Base representa, hasta el momento, la opción más sólida para la estimación automática del Body Condition Score (BCS) de vacas lecheras. Este modelo logró cumplir de forma consistente los criterios de éxito planteados en la etapa de minería de datos, alcanzando métricas clave como un $MAE \leq 0.4$ y un coeficiente de determinación R^2 superior al 70%, tanto en validación como en prueba. Aunque ConvNeXt no presenta la convergencia más suave entre entrenamiento y validación, su capacidad real de generalización supera a las otras arquitecturas evaluadas, lo que se refleja en su menor error y mayor estabilidad predictiva. Asimismo, el análisis de robustez permitió identificar que los errores más frecuentes se asocian a fallos en el recorte de las imágenes, por lo que se mejoró el preprocesamiento para reducir dicha fuente de variabilidad.

Desde la perspectiva del negocio, estos resultados implican que el modelo es capaz de apoyar la estimación temprana del estado de salud de las vacas del corral seis del CAETEC, contribuyendo a la detección oportuna de deterioros en condición corporal, optimización del manejo nutricional y apoyo en la toma de decisiones diarias..

Aunque todavía no se cuenta con la valoración formal de un experto veterinario o zootecnista, este estudio reconoce la necesidad de dicha revisión para validar externamente la utilidad del modelo en un entorno operativo real. La ausencia de esta opinión se justifica debido a que el proyecto se encuentra en una fase experimental centrada en la evaluación técnica; sin embargo, se considera prioritaria para la siguiente etapa, ya que permitiría contrastar la interpretación del modelo con el conocimiento clínico y productivo del ganado.

En cuanto a la revisión contra la base de conocimiento existente, el comportamiento del modelo es coherente con literatura previa, donde arquitecturas más modernas como EfficientNet o ConvNeXt muestran mejor generalización con conjuntos de datos moderados, mientras que modelos tradicionales como VGG tienden al sobreajuste cuando se dispone de pocas imágenes. En este sentido, no se identificaron patrones contradictorios; por el contrario, el modelo confirmó que las regiones

anatómicas con mayor relevancia visual (grupa, lomo, inserción de cola) son determinantes para la estimación del BCS, lo que refuerza el conocimiento técnico previo.

Respecto a la confianza del modelo, las métricas de desempeño indican una consistencia aceptable, especialmente por el bajo sesgo (bias cercano a cero) y la estabilidad entre validación y prueba. No obstante, se reconoce que la confianza no es homogénea en todos los casos: imágenes con recortes erróneos, iluminación extrema o variaciones morfológicas particulares pueden incrementar el error, lo que debe considerarse al interpretar los resultados en campo.

Finalmente, el potencial de despliegue del modelo es alto debido a que ConvNeXt presenta buena escalabilidad, eficiencia en inferencia y compatibilidad con técnicas futuras de transfer learning. Esto permite proyectar su integración en sistemas móviles de registro en establos, aplicaciones de monitoreo continuo o herramientas internas del CAETEC para la evaluación periódica del hato lechero. No obstante, para su implementación final se requerirán pasos adicionales como estandarización de captura, validación experta y pruebas piloto en condiciones reales.

En conjunto, el modelo ConvNeXt-Base ofrece una alternativa viable y prometedora para apoyar la estimación automatizada del BCS, con un desempeño sólido en las métricas técnicas y un valor potencial evidente para los procesos del CAETEC, sin sustituir la experiencia del profesional, sino complementando de manera eficiente.

Referencias

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). *A ConvNet for the 2020s*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 11976–11986. <https://doi.org/10.1109/CVPR52688.2022.01167>